Tokenization, Word Embeddings, Attention & Transformers

Q1. Conceptual - Need for Tokenization

Why neural networks cannot process raw text:

- Neural networks are essentially giant math calculators; they only understand and process numbers, not letters or words.

Tokenization: This is the process of chopping up a sentence into smaller, manageable pieces called "tokens".

Vocabulary: Think of this as the model's personal dictionary, containing a list of all the unique tokens it has learned.

Token IDs: Since models need numbers, every token in the vocabulary is assigned a unique integer (ID).

Character-level vs Word-level:
Character-level tokenization chops text into individual letters (like "c", "a", "t"), while word-level chops text into whole words (like "cat")

Tokenization & Embeddings Example

Q2. Numerical - Tokenization Example

Sentence:

- "Deep learning models learn patterns"

a) Convert sentence into token IDs:

- Step 1: Look up "Deep" in the table → ID is 1.
- Step 2: Look up "learning" → ID is 2.
- Step 3: Look up "models" → ID is 3.
- Step 4: Look up "learn" → ID is 4.
- Step 5: Look up "patterns" → ID is 5.

Final Answer: [1, 2, 3, 4, 5].

b) Shape of embedding matrix for this sentence:

- The sentence has 5 words.
- Each word is turned into a list of 4 numbers (embedding dimension).

Final Shape: 5 × 4 (or a matrix with 5 rows and 4 columns).

c) Why embeddings over one-hot vectors:

- One-hot vectors are mostly filled with useless zeros and take up a massive amount of memory.
- Embeddings are much smaller, packed with numbers, and actually understand what the words mean.

Word Embeddings & Context in NLP

Q3. Conceptual - Word Embeddings

What are they: Word embeddings are lists of numbers (dense vectors) that represent the meaning of a word.

Semantic similarity: Words that mean similar things (like “happy” and “joyful”) will have number lists that look very similar to each other.

Contextual meaning: Advanced embeddings look at the surrounding words to figure out exactly how a word is being used in a specific sentence.

Why they are better: They save a lot of computer memory and help the AI understand relationships between words.

Q4. Conceptual - Context Problem in NLP

The problem: Traditional embeddings give a word the exact same ID and numbers every time, no matter how it is used.

Example: For the word “bank”, traditional models use the same numbers for a “river bank” (nature) and a “financial bank” (money), which confuses the AI.

The solution: Contextual embeddings read the whole sentence first before assigning numbers, meaning “bank” gets a different set of numbers depending on the context.

Attention Mechanism & Attention Scores

Q5. Conceptual - Attention Mechanism

- **Intuition:** Just like human reading, the attention mechanism lets the AI focus heavily on a few important words rather than treating every word equally.
- **Why it was introduced:**
  - Older models would "forget" the beginning of a long sentence by the time they reached the end.
  - Attention was created to fix this memory loss.
- **How it helps:** It assigns a "score" to every word, telling the model exactly which words in the sentence are most relevant to the current word being translated or processed.

Q6. Numerical - Attention Score Calculation

- **Given:** Q = [1, 2] and K = [2, 1]. Formula is Q · K.
- a) **Compute attention score:**
  - Step 1: Multiply the first numbers: 1 × 2 = 2.
  - Step 2: Multiply the second numbers: 2 × 1 = 2.
  - Step 3: Add them together: 2 + 2 = 4.
- **Final Answer:** The attention score is 4.
- b) **What a larger score represents:**
  A larger score simply means the AI should pay a lot of attention to that specific word because it is highly relevant.
- c) **Why Query and Key are used:**
  Think of a search engine.
  The "Query" is what the model is currently looking for, and the "Key" is the label on the word it is checking.
  Multiplying them sees how well they match.

Self-Attention & Transformer Architecture

Q7. Conceptual - Self-Attention

Self-attention: This is when a sentence looks at itself to figure out how its own words relate to one another.

How it works: A word creates a "Query" for itself and compares it against the "Keys" of every other word in the exact same sentence.

Why it is powerful: Standard RNNs process words one by one
↳ one in a line, losing track over long distances.
Self-attention looks at every word simultaneously, easily connecting the first word to the last word.

Q8. Conceptual - Transformer Architecture

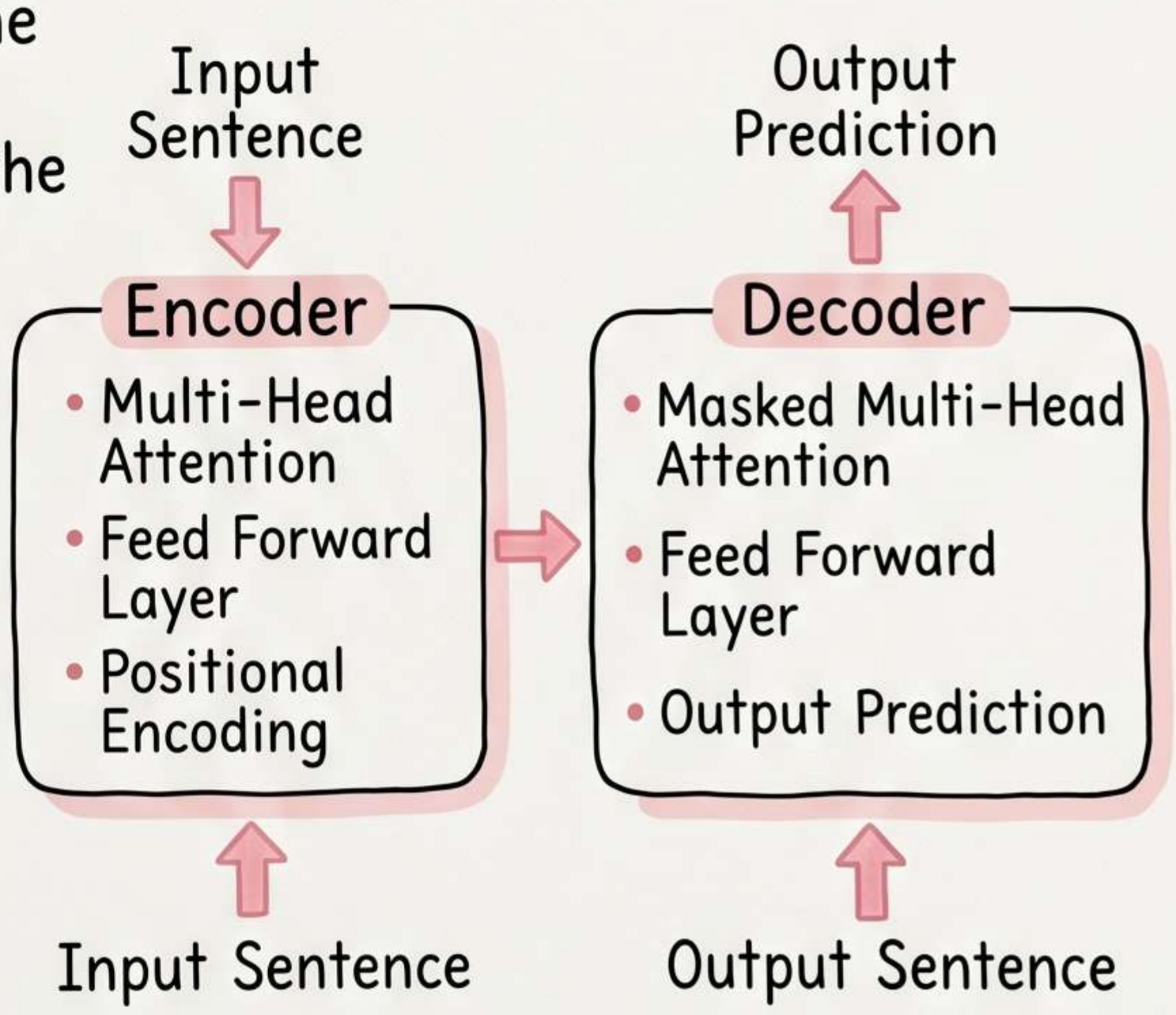
Encoder: The reading section. It analyzes the input sentence and figures out the context and meaning.

Decoder: The writing section. It takes the Encoder's understanding and generates the final output sentence step-by-step.

Multi-head attention: The part that runs several attention mechanisms at the exact same time to look at different relationships.

Feedforward layers: Simple mini-neural networks that process the results of the attention mechanism for each word.

Positional encoding: A digital "stamp" added to words to tell the model what order they came in.



Positional Encoding & Modern NLP Models

Q9. Conceptual - Positional Encoding

Why it is needed:

- Transformers read a whole paragraph at the exact same time in parallel, meaning they have no idea which word comes first or last naturally.

The problem without it:

↳ Without order, the sentence “The dog chased the cat” looks completely identical to “The cat chased the dog” to the model.

Q10. Conceptual + Application - Modern NLP Models

Parallel processing:

- Transformers process all words at once instead of one-by-one, making them incredibly fast.

Long-range dependencies:

- The attention mechanism easily remembers connections between words that are very far apart in a document.

Scalability:

↳ The architecture is easy to stack, meaning researchers can build massive models with billions of parameters.

Training efficiency:

- Because they process things in parallel, they can learn from enormous internet datasets much faster than old models.

One-Hot Encoding vs Embeddings

Q11. Conceptual - One-Hot Encoding vs Embeddings

	One-Hot Encoding	Word Embeddings
Vector size	• Massive (matches total vocabulary size)	• Small and fixed (e.g., 300 numbers)
Semantic meaning	• None (does not underst word relationships)	• High (groups similar words together)
Sparsity	• Very sparse (mostly zeros)	• Dense (mostly meaningful numbers)
Memory efficiency	• Terrible (wastes a lot of memory)	• Excellent (highly efficient)

Why embeddings are preferred:

- They use less memory and actually give the AI an understanding of what words mean.

Q12. Numerical - Embedding Matrix Dimensions

Given:

↳ Vocabulary size = 20,000 and Embedding dimension = 300.

a) Size of the embedding matrix:

- Step 1: The matrix needs one row for every word in the vocabulary (20,000).
- Step 2: The matrix needs a column for every dimension (300).

Final Answer: 20,000 × 300.

b) Trainable parameters:

↳ Step 1: Multiply the rows by the columns.

- Step 2: 20,000 × 300 = 6,000,000.

Final Answer: 6,000,000 parameters.

c) Shape of embedded output for 10 words:

- Step 1: You have 10 words.
- Step 2: Each word gets a vector of size 300.

Final Answer: 10 × 300 (or a shape of 10, 300).

Multi-Head Attention & Encoder vs Decoder

Q13. Conceptual - Multi-Head Attention

- What it is:
  - Instead of looking at a sentence just one way, the model uses multiple attention "heads" to look at the sentence in several different ways at once.
- Why use multiple:
  - Language is complex. One head might not catch all the details.
- How they learn relationships:
  - One head might focus entirely on grammar (verbs vs nouns), while another head focuses entirely on "who is doing the action" to whom.

Q14. Conceptual - Encoder vs Decoder

- Encoder:

→ Encoder: Its main job is Language understanding. It reads text and figures out its meaning.

- Application: Sentiment Analysis (reading a movie review and understanding if it is positive or negative).

- Decoder: Its main job is Text generation. It takes context and writes out new text.

- Application: Story generation (giving the AI a prompt and having it write the next paragraph).

Transformer Use Cases

Q15. Application-Based - Transformer Use Cases

- **Machine translation**: Transformers read a sentence in English, understand the full context, and generate a highly accurate sentence in French.

- **Chatbots**: They use Transformers to remember the whole history of your conversation so they can reply to you with contextually appropriate, human-like answers.

- **Text summarization**: Because they can process massive amounts of text at once, they can scan an entire long document and extract only the most crucial points.

- **Why they are suitable**: Transformers are built to handle context beautifully, they remember long-range details, and they process data efficiently, making them perfect for complex language tasks.